

Constraint-Aware Sequential Design for Risk-Region Identification in Implicit Combinatorial Spaces

隐式组合域中的风险区域识别：约束感知的序贯实验设计

匿名作者

摘要

黑盒决策策略部署到订单履约、动态调度或信用审批等业务流程时，部署规则需要按输入状态决定使用自动策略还是备用流程。该规则取决于各可行输入状态下的失效概率。资源、权限、时序和跨实体一致性约束通常形成规模大、稀疏且不可枚举的组合可行域，而失效结果只能通过具有随机性和运行成本的黑盒实验获得。历史分布抽样可能遗漏测度较小但损失较高的区域；标准水平集估计则通常依赖显式候选集或易于积分的连续输入域。

本文定义有限运行预算下的决策驱动风险区域学习问题。正常运行成本、备用流程成本和失效损失确定状态相关风险阈值及最优部署区域。实验设计在探索新输入状态和复验已有状态之间分配预算，以最小化部署规则相对于完全信息规则的期望后悔。所提出的约束感知逐步不确定性削减方法以约束单元表示可行域，用单元质量度量各单元在参考测度下的权重，并计算单元级后验部署风险界和采集价值界。方法在选定单元内进行条件采样，将点采集值按变量取值、区间和低阶交互聚合为变量条件采集景观，以选择分裂变量和分裂阈值。景观用于确定展开顺序，单元界用于剪枝和停止判定。理论分析考察采集近似误差、后验部署风险界和自适应复验；计算实验包括可枚举合成实例、大规模经典决策策略和交互式 AI Agent。

关键词：序贯实验设计；约束规划；水平集估计；仿真优化；风险区域；选择性部署；黑盒决策策略

1 引言

补货、调度和信用审批策略可能由启发式规则、预测模型、商业求解器、外部服务及人工复核等模块构成；AI Agent 还会通过多轮观察和工具调用改变业务状态。此类策略的输入-输出关系难以从内部结构解析，部署前性能通常通过仿真、沙箱运行或受控实验估计。

平均失效率不足以确定部署规则。例如，某策略在普通订单上成本低且失效率低，但在“部分退款、跨仓拆单、额度刚好越过审批线”等状态组合中可能频繁违反业务规则。此时可在低风险状态使用自动策略，并将高风险状态转交人工或备用流程。因此，部署决策需要估计自动策略与备用流程之间随状态变化的运营边界。

设 x 表示一个完整业务状态, $p(x)$ 表示黑盒策略在该状态下的失效概率。若自动策略的正常成本为 $c_\pi(x)$, 备用方式的成本为 $c_0(x)$, 一次失效的期望损失为 $\ell(x)$, 那么只有当

$$c_\pi(x) + p(x)\ell(x) \leq c_0(x) \quad (1)$$

时, 自动策略在状态 x 上的期望成本不高于备用流程。式 (1) 因而定义自动策略的部署区域, 以及需要由人工审核或备用流程覆盖的状态集合。

$p(x)$ 未知, 且输入状态受到业务约束。库存守恒、权限匹配、任务先后关系以及实体间的容量、额度和一致性要求共同限定可行输入。若原始变量空间为 $\mathcal{D}$, 令 C_j 表示第 j 个业务约束谓词, 则可行输入集合为

$$\mathcal{X} = \{x \in \mathcal{D} : C_j(x) \text{ holds}, j \in \mathcal{J}\}, \quad (2)$$

其中 C_j 可以是代数等式或不等式、逻辑蕴含与析取、表约束, 或 `AllDifferent`、`NoOverlap` 和 `Cumulative` 等可传播的全局约束。对数值约束, 可取 $C_j(x) \iff g_j(x) \leq 0$; 本文不要求 g_j 连续、可微或凸, 也不要求把全局约束分解为标量不等式。那么 $|\mathcal{D}|$ 往往随变量数指数增长, 而 $\mathcal{X}$ 只由约束系统隐式给出。即使求得一个可行输入状态并不困难, 列举全部可行输入状态、计算每个状态的风险或在整个集合上积分仍然不可行。

一次黑盒运行可能涉及长时仿真、远程模型调用或完整业务 workflow。同一输入状态上的独立运行也会因需求扰动、仿真随机数或策略随机性产生不同结果。在有限预算下, 实验设计需要在未观察区域、部署边界附近和已有状态的复验之间分配运行次数。

基于参考业务分布的随机仿真和历史回放通常侧重总体表现; 在有限样本下, 低概率但高损失的状态组合往往难以获得充分覆盖, 这也是稀有事件仿真需要专门采样方法的原因之一 [9]。黑盒安全验证和自适应压力测试通常搜索最坏点、高严重度失效或高似然失效轨迹, 而不直接估计由运营成本诱导的部署区域及其参考测度 [10, 11]。随机水平集估计能够恢复阈值集合, 但其采集函数通常定义在显式有限候选集, 或便于采样、积分和优化的连续输入域上 [2, 4, 5, 6]。约束规划可以传播业务规则并生成可行解, 但不利用随机黑盒运行结果选择下一实验点 [12]。本文研究运营损失、随机黑盒响应和隐式组合可行域同时存在时的序贯实验设计。

研究问题如下:

在由约束系统隐式定义且无法枚举的组合可行域中, 如何利用有限的随机黑盒运行预算, 序贯选择输入状态与复验次数, 从而学习由运营成本诱导的最优策略部署区域, 并最小化预算耗尽后的部署后悔?

固定候选池上的标准主动学习不能控制池外误差。全局逐步不确定性削减准则需要聚合整个可行域的后验部署风险, 并计算各候选运行带来的期望风险下降。当 $\mathcal{X}$ 不可枚举时, 风险聚合、采集值计算和采集函数最大化都需要在隐式组合域上完成。预先抽取的固定候选池既不能度量池外遗漏的参考测度质量, 也不能直接给出完整可行域上的风险界。

本文用约束搜索树表示序贯实验设计的计算域。树节点是由部分赋值和传播后变量域定义的约束单元 U 。约束传播删除不可行分支, 精确或近似计数给出单元质量, 后验模型给出单元部署风险界和采集价值界。算法在需要继续细分的单元内条件采样, 将点采集值按变量取值、区

间和低阶交互聚合，据此选择分裂变量和分裂阈值。分支定界优先展开采集上界较高的单元，并依据有效界剪枝；具体输入状态仅在需要进行点级评价时实例化。

本文的研究内容包括四部分。第一，定义隐式组合可行域上的决策驱动风险区域学习模型，由正常运行成本、备用流程成本和失效损失确定状态相关部署阈值，并以完全信息规则为基准定义部署后悔。第二，构造约束感知逐步不确定性削减方法，将约束单元、约束传播和单元质量用于全局后验部署风险及采集函数的计算。第三，构造分支定界采集搜索：单元界用于剪枝和误差控制，变量条件采集景观用于选择分裂；搜索同时处理新状态探索和已有状态复验。第四，设计涵盖可枚举合成问题和大规模业务环境的计算实验，与随机可行采样、固定候选池水平集估计、压力测试方法及消融版本比较。

本文将黑盒策略定义为能够在给定输入状态下运行并返回可判定随机结果的策略，包括启发式调度器、学习型库存策略、仿真控制器、商业规则引擎和 LLM Agent。C-SUR 只要求约束系统能够在部分赋值下执行增量传播；状态生成、策略运行、轨迹记录和失效判定由外部实验系统完成。

2 相关研究与研究缺口

2.1 随机水平集估计与序贯仿真

水平集估计关注从有限观测中恢复 $\{x : f(x) \geq a\}$ 。Gotovos et al. [2] 用 Gaussian process (GP) 置信界同时推进采样与集合分类；Bogunovic et al. [3] 以截断方差削减统一处理水平集估计、点成本和异方差噪声。在可靠性和计算机实验中，Bect et al. [4] 从 Bayes 决策角度推导逐步不确定性削减 (SUR) 准则以估计失效集合的测度，Chevalier et al. [5] 进一步研究其批量化与快速计算。针对随机响应，Lyu et al. [6] 系统比较了带噪水平集估计中的代理模型与序贯设计。本文沿用这些工作中的后验集合风险和采集价值定义。

这些方法通常在有限候选集、规则网格或便于采样、积分和连续优化的欧氏集合上计算。实际 SUR 需要反复近似全域积分并优化采集函数，其计算成本已有专门研究 [5]。本文所考虑的 $\mathcal{X}$ 由混合逻辑和整数约束隐式定义，且不能预先充分离散化；部署损失聚合和下一运行点搜索均在该域上进行。仅用约束系统过滤候选点无法控制固定候选池以外的遗漏质量。

离散仿真优化中的 optimal computing budget allocation 研究如何在有限备选方案之间分配复验次数，以提高选择正确方案的概率 [7]。Song [8] 在输入模型不确定性下定义候选解的 Bayesian risk set，并序贯选择需要仿真的“解-输入模型”组合。该 risk set 不同于本文按状态失效概率定义的部署区域；两类模型都以终端集合损失评价复验分配。这些方法通常从显式有限备选集出发。隐式约束域还需要联合决定搜索哪些未观测区域以及在已有状态上追加多少次运行。

2.2 压力测试、可靠性与失效发现

可靠性分析和稀有事件仿真关注失效概率的高效估计。例如，subset simulation 将小概率失效事件分解为一系列较常见的条件事件 [9]。黑盒安全验证还包括 falsification、最可能失效搜索和失效概率估计等不同任务；Corso et al. [10] 对优化、路径规划、强化学习与重要性采样

路线作了系统综述。Adaptive stress testing 进一步把可能失效轨迹的搜索形式化为序贯控制问题 [11]。

这些方法用于发现系统薄弱点、搜索高严重度或高似然失效轨迹，或者估计聚合失效概率。其目标函数通常不包含状态相关部署区域的集合损失。点式搜索还可能在同一失效机制附近重复取样。本文的损失函数按参考测度和失效代价加权区域误判，因此与最坏点搜索和整体失效概率估计对应不同的决策目标。

2.3 约束规划与受约束测试

约束规划以原生逻辑、整数和全局约束表示组合可行域，并通过传播与搜索避免展开全部赋值；Rossi et al. [12] 给出了该领域的系统总结，Régim [13] 对 AllDifferent 的过滤算法则展示了保留全局约束结构为何能显著增强传播。与本文的单元质量和条件生成接口直接相关，hashing-based 方法已经能够对大型 Boolean 约束进行带置信保证的近似模型计数 [14] 和近似均匀的可满足赋值生成 [15]。这些成果提供了实现 mass 与 sample 可行域操作的候选技术，但本身不利用随机黑盒运行结果决定下一次昂贵实验。

组合交互测试在约束存在时通常构造 constrained covering array，以覆盖所有可行的低阶参数交互；Wu et al. [16] 综述了约束表示与测试生成技术。Constrained locating array 在可行测试中加入故障交互定位能力 [17]。这类方法主要优化测试套件规模、交互覆盖或确定性故障定位。本文还包含状态相关随机失效概率、非均匀参考测度和不对称部署损失，并需要约束求解器为风险聚合、采集搜索和停止判定提供单元表示、单元质量与有效界。

2.4 研究缺口

现有研究分别给出了带噪水平集的序贯统计准则、有限备选集上的复验分配、黑盒失效搜索，以及组合可行域的传播、计数和测试生成。本文考察的计算问题是：在不枚举可行输入状态的前提下，将约束传播和单元质量纳入全局后验部署风险与序贯采集计算，并用部署后悔评价终端集合误差。该问题要求在同一算法中处理随机响应、组合可行域和状态相关部署损失。

主要基线先由约束求解器生成固定可行候选池，再在池内运行标准 SUR。C-SUR 与该基线比较全域加权部署后悔、达到给定采集精度所需的计算量，以及完整可行域上的后验部署风险界。

3 决策模型

本章建立从黑盒运行到部署决策的基本关系。对每个可行输入状态，黑盒策略都有一个未知的失效概率；运营成本把这一概率转化为“使用自动策略还是备用流程”的选择标准。由于不可能在所有状态上无限次运行，实验只能在有限预算内逐步收集信息，并尽量减少最终选错部署动作所造成的损失。下文先说明失效概率如何由随机运行产生，再推导运营成本对应的最优部署决策，最后定义有限预算下的序贯学习目标。

3.1 可行状态与失效概率

一次黑盒运行以完整的初始业务状态 $x = (x_1, \dots, x_d)$ 作为输入。其中的变量可以是布尔、整数、类别、集合或时序对象。只有满足式 (2) 中全部业务约束的状态才允许作为运行输入，这类状态称为可行输入状态。本文用 $\mathcal{X}$ 表示所有可行输入状态组成的集合，即式 (2) 定义的可行域。

给定黑盒策略 π ，一次运行不仅产生最终结果，还会产生由观察、动作、环境反馈和状态变化组成的完整轨迹。本文将该轨迹记为

$$\tau(x; \pi, \omega) = ((s_k, o_k, a_k, e_k)_{k=0}^{K-1}, s_K, o_K), \quad (3)$$

其中， $s_0 = x$ 是初始状态。在第 k 步，环境处于状态 s_k ，策略获得观察 o_k 并采取动作 a_k ；环境执行该动作后返回反馈 e_k ，再进入状态 s_{k+1} 。运行经过 K 步后终止于状态 s_K ，并产生终止观察 o_K 。 ω 表示策略、仿真器和外部环境中的全部随机因素。反馈 e_k 可以是工具返回值、状态变化或事件记录；本文不要求环境提供强化学习意义上的数值奖励。

失效判定规则 F 预先给定。每次运行结束后， F 检查完整轨迹是否达到终端目标、是否在过程中违反业务规则，以及是否发生运行错误，并据此输出二元标签：

$$Y(x, \omega) = F(\tau(x; \pi, \omega)) \in \{0, 1\}, \quad p(x) = \mathbb{P}\{Y(x, \omega) = 1 \mid x\}. \quad (4)$$

其中， $Y = 1$ 表示本次运行失效， $Y = 0$ 表示成功； $p(x)$ 是策略在输入状态 x 上的失效概率。判定规则 F 本身是确定的：给定同一条完整轨迹，它始终返回相同标签。运行过程则可以是随机的，因此同一个可行输入状态经过多次运行，可能得到不同的轨迹和标签。

输入可行性和运行失效是两个不同的问题。业务约束确定一个状态是否属于 $\mathcal{X}$ ，却不能直接给出被测策略在该状态上的失效概率 $p(x)$ 。本文将 $p(x)$ 视为未知量。为了使这个概率在研究期间保持稳定，本文假设被测策略、判定规则以及仿真环境的版本和配置均保持不变。

3.2 失效概率与最优部署决策

在每个输入状态 x 上，部署者需要选择使用自动策略 π ，还是转交人工或其他备用流程。令 $z(x) \in \{0, 1\}$ 表示这一选择： $z = 1$ 表示使用自动策略， $z = 0$ 表示使用备用流程。

两种选择的成本由三个量决定。 $c_\pi(x)$ 是自动策略的基础运行成本， $c_0(x)$ 是备用流程的成本， $\ell(x) > 0$ 是自动策略发生失效时造成的额外期望损失。本文将这三个量视为部署者预先给定的已知信息。给定真实失效概率 $p(x)$ ，在状态 x 上采取部署动作 z 的期望成本为

$$C(x, z; p) = \begin{cases} c_0(x), & z = 0, \\ c_\pi(x) + p(x)\ell(x), & z = 1. \end{cases} \quad (5)$$

式 (5) 只是把两种部署选择的成本放在一起：使用备用流程时支付 $c_0(x)$ ；使用自动策略时支付基础运行成本，并承担失效造成的期望损失。比较这两项成本，可以得到状态 x 上的最大可接受失效概率，以及完全知道 $p(x)$ 时的最优部署规则：

$$\alpha(x) = \frac{c_0(x) - c_\pi(x)}{\ell(x)}, \quad z^*(x) = \mathbf{1}\{p(x) \leq \alpha(x)\}. \quad (6)$$

这里, $z^*(x)$ 表示在完全知道 $p(x)$ 时的最优选择; 指示函数 $\mathbf{1}\{\cdot\}$ 在括号内条件成立时取 1, 否则取 0。因此, 当 $p(x) \leq \alpha(x)$ 时选择自动策略, 否则选择备用流程。

阈值 $\alpha(x)$ 不一定落在概率区间 $[0, 1]$ 内。若 $\alpha(x) < 0$, 备用流程在该状态下始终更便宜; 若 $\alpha(x) > 1$, 自动策略始终更便宜。只有当 $0 < \alpha(x) < 1$ 时, 最优选择才真正取决于未知的 $p(x)$ 。当 $\alpha(x) = 0$ 或 1 时, 两种选择可能在概率区间的端点处成本相同。

将这一逐状态决策应用到整个可行域, 可以得到最优自动部署区域 $\mathcal{D}^*$:

$$\mathcal{D}^* = \{x \in \mathcal{X} : p(x) \leq \alpha(x)\}. \quad (7)$$

其补集 $\mathcal{X} \setminus \mathcal{D}^*$ 是不宜自动部署的高风险区域。这里的“高风险”由运营成本决定, 并不只是指失效概率最大的状态。若所有状态共享同一个阈值 α , 式 (7) 就成为标准的固定阈值水平集。状态相关阈值则允许不同状态采用不同的风险容忍度。例如, 在 $c_0(x) - c_\pi(x)$ 不变时, 失效损失 $\ell(x)$ 越大, $\alpha(x)$ 越低, 自动部署条件也越严格。

为了汇总整个可行域上的部署损失, 本文用参考测度 μ 为各状态或状态区域赋予权重。这些权重可以来自实际部署频率, 也可以反映情景的重要性或管理者指定的压力测试重点。当 $\mathcal{X}$ 是有限集合时, 关于 μ 的积分可以理解为对所有可行状态的加权求和。

沿用统计决策理论中以候选规则相对于最优规则的额外期望损失衡量决策质量的思想 [1], 本文将任意部署规则 z 相对于完全信息最优规则 z^* 多产生的成本称为部署后悔:

$$\mathcal{L}(z; p) = \int_{\mathcal{X}} [C(x, z(x); p) - C(x, z^*(x); p)] d\mu(x). \quad (8)$$

方括号中的差值表示: 在状态 x 上采用规则 z , 相对于最优规则 z^* 多付了多少期望成本。如果二者选择相同动作, 该差值为零; 如果选择不同动作, 则

$$C(x, z(x); p) - C(x, z^*(x); p) = |p(x) - \alpha(x)|\ell(x). \quad (9)$$

其中, $|p(x) - \alpha(x)|$ 表示真实失效概率偏离部署阈值的程度, 乘以失效损失 $\ell(x)$ 后, 就得到选错动作所增加的期望成本。在两种动作成本相同的边界点上, 即使动作不同, 该项仍为零。最后, 参考测度 μ 将各状态上的额外成本汇总为总体部署后悔。

这一指标与普通分类错误率不同。分类错误率只统计有多少状态被分错, 部署后悔还考虑各区域在参考测度 μ 下的权重、状态相关的失效损失 $\ell(x)$, 以及真实失效概率到部署阈值的距离 $|p(x) - \alpha(x)|$ 。因此, 两个状态即使都选择了错误的部署动作, 造成的后悔也可能不同。特别地, 靠近边界的状态虽然更难判断, 但选错动作所增加的成本通常也更小。

3.3 有限预算下的序贯学习

失效概率 $p(x)$ 未知, 每次黑盒运行只能在一个输入状态上产生一个失效标签, 并且会消耗时间、算力或调用费用。本文用已知的 $c(x) > 0$ 表示在状态 x 上完成一次运行的成本。这里的 $c(x)$ 是实验成本, 不同于第 3.2 节中的自动策略部署成本 $c_\pi(x)$ 和备用流程成本 $c_0(x)$ 。由于运行预算有限, 实验需要利用已有结果决定下一次测试哪个状态, 以及是否复验已经测试过的状态。

完成 t 次运行后, 按照观测顺序记录各次运行的输入状态和失效标签, 得到实验历史

$$\mathcal{H}_t = ((x_i, Y_i))_{i=1}^t. \quad (10)$$

历史采用有序序列，是因为较早的观测会影响后续选择。用 $\mathcal{A}$ 表示序贯选点策略。在预算尚未耗尽时， $\mathcal{A}$ 根据当前历史 $\mathcal{H}_t$ 选择下一个可行输入状态 $x_{t+1} \in \mathcal{X}$ ；对于确定性策略，可以写成 $x_{t+1} = \mathcal{A}(\mathcal{H}_t)$ ，本文也允许策略使用随机选点。如果该状态尚未测试，这次选择属于探索；如果已经测试，则属于复验。 $\mathcal{A}$ 是规定如何选点的决策规则，按照该规则实际产生的输入和标签序列才是随机实验过程。

选定 x_{t+1} 后，环境重置到该输入状态，并使用新的独立随机源 ω_{t+1} 完成运行。新得到的标签满足

$$Y_{t+1} \mid (\mathcal{H}_t, x_{t+1}) \sim \text{Bernoulli}(p(x_{t+1})). \quad (11)$$

式 (11) 表示：给定本次选择的输入状态后，标签服从以 $p(x_{t+1})$ 为参数的伯努利分布，不再依赖过去的运行结果。过去的结果仍会通过 $\mathcal{H}_t$ 影响 x_{t+1} 的选择。因此，自适应实验产生的标签序列一般不是无条件独立的。

设总运行预算为 B 。选择 x_{t+1} 时，还必须满足已用成本与本次成本之和不超过 B 。由于 $c(x)$ 可以随输入状态变化，固定预算下能够完成的运行次数也不是预先给定的。记预算内实际完成的运行次数为 N_B ，则

$$\sum_{i=1}^{N_B} c(x_i) \leq B. \quad (12)$$

参考测度 μ 和 $\mathcal{A}$ 产生的选点分布具有不同作用。 μ 决定部署后悔中各状态的权重； $\mathcal{A}$ 的选点分布则决定预算用于哪些状态。为了更快辨别部署区域， $\mathcal{A}$ 可以增加对罕见状态或部署边界附近状态的测试，因此不要求其选点分布与 μ 相同。

预算结束后，根据最终历史 $\mathcal{H}_{N_B}$ 形成部署规则 $\hat{z}_B : \mathcal{X} \rightarrow \{0, 1\}$ ，其中下标 B 表示该规则是在预算 B 下学到的。 $\mathcal{A}$ 决定如何收集信息， $\hat{z}_B$ 则利用收集到的信息决定各状态采用自动策略还是备用流程。二者不是同一个对象； $\mathcal{A}$ 通过改变最终历史而影响 $\hat{z}_B$ 。下一节将具体说明如何根据后验部署风险形成这一部署规则。

问题 1 (决策驱动的约束风险区域学习) 失效概率 $p(x)$ 只能通过随机运行估计，可行域 $\mathcal{X}$ 只能通过业务约束及其传播接口访问，且无法预先枚举。给定总预算 B ，选择序贯选点策略 $\mathcal{A}$ ，使预算结束时得到的部署规则 $\hat{z}_B$ 的期望部署后悔最小：

$$\min_{\mathcal{A}} \mathbb{E}[\mathcal{L}(\hat{z}_B; p)] \quad \text{s.t.} \quad x_t \in \mathcal{X} \quad (t = 1, \dots, N_B), \quad \sum_{t=1}^{N_B} c(x_t) \leq B. \quad (13)$$

虽然 $\mathcal{A}$ 没有显式写在损失函数内部，但不同的选点策略会产生不同的最终历史，因而得到不同的 $\hat{z}_B$ 。式 (13) 中的期望覆盖各次运行的随机结果；如果 $\mathcal{A}$ 本身采用随机选点，也覆盖这部分随机性。真实的失效概率函数 p 在整个实验期间保持固定，但对实验者未知。

该目标不同于几类常见任务。最大化 $p(x)$ 只寻找最容易失效的状态；最大化独特失效数需要预先定义失效类型；估计整体平均失效率只关心一个聚合概率。这些目标都不直接决定每个状态应使用自动策略还是备用流程，因此不等价于最小化部署后悔。

4 完全枚举下的 SUR 基准与组合计算障碍

第 3 节定义了预算结束后需要最小化的部署后悔，但尚未说明如何选择下一次运行。已有的逐步不确定性削减 (SUR) 方法以当前后验风险为起点，并优先选择能够使下一步期望后验风险下降最多的实验点 [4, 5, 6]。本节沿用这一基本思想，但将其中的后验风险改为第 3 节部署后悔所诱导的风险。因此，下面的一步采集框架来自已有 SUR 文献，具体的局部部署风险则由本文的状态相关阈值和失效损失推导得到。

已有水平集估计方法通常在显式有限候选集上计算采集函数，或者假设输入域便于采样、积分和连续优化 [2, 4, 6]。这并不意味着 SUR 理论普遍要求整个输入域可枚举。本文采用“ $\mathcal{X}$ 能够完全枚举”这一更强的理想化条件，是为了精确计算所有状态的后验风险和采集值，从而得到不含搜索近似的参照结果。本节先在这一条件下给出完整的 SUR 基准；随后说明，当 $\mathcal{X}$ 只能通过约束系统隐式访问时，风险聚合和采集搜索面临哪些计算障碍。该基准不是本文提出的 C-SUR 方法，而是后文构造 C-SUR 和评价其采集精度的参照。

4.1 由部署后悔得到的后验风险与 SUR 基准

给定实验历史 $\mathcal{H}_t$ ，统计模型为未知函数 p 提供后验分布。本文不在决策模型中限定具体的代理模型；它可以是适用于伯努利观测的高斯过程分类模型、分层模型，或其他能够进行后验预测与更新的模型。本节只使用该后验回答两个问题：当前应如何部署，以及在哪个状态上增加一次运行最有价值。

在历史 $\mathcal{H}_t$ 给定后， $p(x)$ 仍然未知，因此无法直接判断两种部署动作中哪一个成本更低。若选择自动策略，而真实情况应当使用备用流程，则局部后悔为 $\ell(x)(p(x) - \alpha(x))_+$ ；反之，若选择备用流程，而真实情况适合自动部署，则局部后悔为 $\ell(x)(\alpha(x) - p(x))_+$ 。对当前后验分布取期望，得到两种动作各自的后验局部后悔：

$$\rho_t^{\text{auto}}(x) = \ell(x)\mathbb{E}[(p(x) - \alpha(x))_+ | \mathcal{H}_t], \quad (14)$$

$$\rho_t^{\text{back}}(x) = \ell(x)\mathbb{E}[(\alpha(x) - p(x))_+ | \mathcal{H}_t], \quad (15)$$

其中 $(u)_+ = \max\{u, 0\}$ 。在状态 x 上，当前最合理的动作是选择后验局部后悔较小的一项；等价地，当 $\mathbb{E}[p(x) | \mathcal{H}_t] \leq \alpha(x)$ 时选择自动部署。采取较优动作后仍然无法消除的后验不确定性记为

$$r_t(x) = \min\{\rho_t^{\text{auto}}(x), \rho_t^{\text{back}}(x)\}, \quad (16)$$

将 $r_t(x)$ 按参考测度 μ 汇总，得到整个可行域上的后验部署风险

$$R_t = \int_{\mathcal{X}} r_t(x) d\mu(x). \quad (17)$$

在标准 SUR 框架中，下一实验点按照一次新观测能够带来的期望后验风险下降来选择 [4, 5]。将这一原则应用于 R_t ，并用运行成本 $c(x)$ 归一化，得到完全枚举条件下的单位成本一步采集函数

$$a_t(x) = \frac{R_t - \mathbb{E}[R_{t+1} | \mathcal{H}_t, \text{在 } x \text{ 处运行一次}]}{c(x)}. \quad (18)$$

分子表示在 x 上增加一次运行预计能够减少多少全局后验风险，分母是这次运行的成本。因此， $a_t(x)$ 越大，单位实验成本带来的预期风险下降越多。完全枚举基准选择 $a_t(x)$ 最大的状态作为下一实验点。

4.2 隐式组合域中的三个计算障碍

上述基准要求遍历或有效优化整个 $\mathcal{X}$ 。当 $\mathcal{X}$ 只能通过组合约束隐式访问时，会遇到三个直接障碍。首先，计算 R_t 需要汇总所有可行状态的后验风险。其次，评价一个候选点需要预测该次观测对整个可行域的影响。最后，即使能够近似算出若干点的 $a_t(x)$ ，仍需在混合逻辑和整数约束下找到全局最大值。

固定候选池方法预先给定或采样 $S \subset \mathcal{X}$ ，并将分类和采集函数最大化限制在 S 上；这类计算方式常见于 pool-based 水平集估计 [2, 6]。其近似误差包括池外高风险区域的遗漏，以及候选池经验分布与参考测度 μ 的偏差。扩大候选池会增加代理预测和采集计算成本，但不能直接给出池外遗漏质量的上界。

5 约束感知的逐步不确定性削减方法

本文提出约束感知逐步不确定性削减方法 (Constraint-Aware Stepwise Uncertainty Reduction, C-SUR)。它不预先列出所有可行输入状态，而是用一组约束单元逐步划分 $\mathcal{X}$ 。每个单元覆盖一批满足相同局部条件的可行状态，算法先估计该单元的参考测度质量以及其中可能达到的风险和采集价值。只有当一个单元仍可能包含有价值的实验点时，才继续将它细分。单元内部应当按哪个变量分裂，则由少量条件样本所呈现的采集值差异决定。通过这种方式，C-SUR 把约束传播、全局风险计算和下一实验点搜索连接起来，同时避免完整枚举 $\mathcal{X}$ 。

5.1 用约束单元表示不可枚举空间

约束单元 U 由部分变量赋值和其余变量当前允许的取值范围构成。它所覆盖的完整可行状态集合记为

$$\mathcal{X}(U) = \{x \in \mathcal{X} : x \text{ 与 } U \text{ 中的部分赋值一致}\}. \quad (19)$$

例如，一个单元可以固定审批状态，同时只限制库存变量的上下界，而不必指定完整订单状态。本文保留业务约束原有的谓词形式和全局结构，不要求将其全部拆成标量不等式。这样，约束求解器在输入状态尚未完全确定时，就能排除不可能的变量取值。以 `AllDifferent` 为例，原生全局约束通常比一组两两不等式更早发现冲突。传播越早缩小变量范围，后续需要展开的搜索分支通常越少。

根单元不固定任何额外条件，因此覆盖整个 $\mathcal{X}$ 。选择一个尚未固定的变量或阈值进行分支，会得到若干互不相交且共同覆盖父单元的子单元。每次产生新单元后，`propagate(U)` 传播当前条件的逻辑后果，并删除满足 $\mathcal{X}(U) = \emptyset$ 的空单元；`split(U)` 表示上述分裂操作。

为了知道一个单元在最终部署风险中占多大权重，算法还需要计算

$$M(U) = \mu(\mathcal{X}(U)), \quad (20)$$

称为单元质量。 $\text{mass}(U)$ 可以通过精确或近似加权模型计数、重要性采样或领域特定动态规划得到 $M(U)$ 。若只能近似计算, 就同时保留误差区间。只有在需要评价具体实验点时, $\text{sample}(U)$ 才从 $\mathcal{X}(U)$ 中生成完整可行状态。

C-SUR 并不依赖某一种约束求解器。它只要求底层系统能够传播部分赋值、查询传播后的变量域、复制分支状态并判断单元是否为空。单元质量估计和完整状态生成可以由独立的计数与采样模块实现。

5.2 单元级部署风险与采集价值界

单元通常仍包含大量状态, 无法逐点计算风险。为此, 算法对每个当前单元 U 维护它对全局后验部署风险贡献的上下界:

$$\underline{R}_t(U) \leq \int_{\mathcal{X}(U)} r_t(x) d\mu(x) \leq \bar{R}_t(U), \quad (21)$$

其中 $r_t(x)$ 是第 4.1 节定义的局部后验部署风险。下界和上界分别表示该单元至少和至多贡献多少全局风险。具体构造可以利用单元质量 $M(U)$ 、单元内后验均值和方差的范围、代理模型的正则性, 以及质量估计误差。

同样地, 算法用下面的区间包住该单元内最佳实验点的采集价值:

$$\underline{a}_t(U) \leq \max_{x \in \mathcal{X}(U)} a_t(x) \leq \bar{a}_t(U). \quad (22)$$

若某个单元的采集上界已经低于其他单元中已知点的采集值, 就不必继续搜索该单元。反之, 上界较大的单元应当优先细分。与只在生成候选点时检查约束的固定候选池方法相比, 这里的约束单元还直接参与风险汇总、搜索排序和停止判定。

5.3 变量条件采集景观与自适应分裂

采集上界可以指出哪个单元值得继续搜索, 却不能说明应当按哪个变量分裂。固定变量顺序、最小域优先等常见约束规划规则主要为了尽快找到可行解, 并不考虑哪些区域更有实验价值。C-SUR 因此在待展开单元 U 中生成少量完整状态, 观察点采集值如何随变量取值变化。本文将这种条件关系称为变量条件采集景观 (variable-conditioned acquisition landscape)。

设 m_U 是本轮在单元 U 中生成的条件样本数。这些样本来自分布 ν_U , 且都满足当前单元和全部业务约束:

$$x^{(1)}, \dots, x^{(m_U)} \sim \nu_U, \quad \nu_U(\mathcal{X}(U)) = 1 \quad (23)$$

对每个样本, 算法计算点采集值, 或者使用较便宜的近似 $\tilde{a}_t(x^{(k)})$ 。若采样分布 ν_U 与参考测度在该单元内的条件分布不同, 就用重要性权重 w_k 校正。对于类别变量或有限域变量 x_j , 固定其取值为 v 后的平均采集值估计为

$$\hat{G}_{t,j,v}(U) = \frac{\sum_{k=1}^{m_U} w_k \tilde{a}_t(x^{(k)}) \mathbf{1}\{x_j^{(k)} = v\}}{\sum_{k=1}^{m_U} w_k \mathbf{1}\{x_j^{(k)} = v\}}, \quad (24)$$

分子累加满足 $x_j = v$ 的样本采集值, 分母是这些样本的总权重, 因此 $\hat{G}_{t,j,v}(U)$ 可以理解为“在当前单元内进一步固定 $x_j = v$ 后, 平均实验价值有多大”。对于整数或连续变量, 候选阈值 b

把样本分为 $x_j \leq b$ 和 $x_j > b$ 两组；对于取值很多的有限域变量，可以先按约束结构或后验相似性合并取值。样本量足够时，也可以用同样方法考察两个变量取值组合的影响，但不要求为所有高阶组合建立统计量。

考虑一个候选分裂 $\mathcal{S}(U) = \{U_1, \dots, U_K\}$ 。用 $\widehat{G}_t(U_h)$ 表示落入子单元 U_h 的样本加权平均采集值， $\widehat{G}_t(U)$ 表示全部父单元样本的加权平均采集值， $\widehat{M}(U_h)$ 表示子单元的质量估计。下面的分数衡量各子单元的平均采集值是否存在明显差异：

$$H_t(\mathcal{S} | U) = \sum_{h=1}^K \widehat{M}(U_h) \left[\widehat{G}_t(U_h) - \widehat{G}_t(U) \right]^2, \quad (25)$$

若所有子单元的平均采集值接近父单元， H_t 较小，这次分裂对区分高价值区域帮助有限；差异越明显， H_t 越大。实际选择分裂时，还要考虑分裂能否收紧单元界，以及是否产生质量极不均衡的子单元：

$$\text{SplitScore}_t(\mathcal{S} | U) = \lambda_H H_t(\mathcal{S} | U) + \lambda_B \widehat{\Delta}_t^{\text{bound}}(\mathcal{S}) + \lambda_M \text{Balance}(\mathcal{S}), \quad (26)$$

其中， $\widehat{\Delta}_t^{\text{bound}}(\mathcal{S})$ 估计分裂后单元界能够收紧多少， $\text{Balance}(\mathcal{S})$ 奖励质量较为均衡的分裂，非负权重 $\lambda_H, \lambda_B, \lambda_M$ 控制三项的相对作用。分数只对约束传播后仍非空的子单元计算。

变量条件景观只决定如何分裂，不直接用于剪枝或误差认证。原因是有限条件样本可能漏掉狭窄但重要的区域， $\tilde{a}_t$ 也可能带有近似误差。所有剪枝、采集差距和后验风险证书仍以有效的单元上下界为依据。若同一单元后来再次进入搜索前沿，可以追加条件样本并更新分裂分数。

5.4 分支定界式采集搜索

在第 t 轮选点时，C-SUR 从覆盖整个可行域的根本单元开始，维护尚待检查的单元集合 $\mathcal{Q}_t$ ，称为采集搜索前沿。算法优先检查采集上界 $\bar{a}_t(U)$ 最大的单元，因为当前最优实验点最可能位于其中。

每次检查一个单元时，算法先传播约束并删除空单元。若该单元的采集上界不超过当前最佳点的采集值，就可以确定它不可能改进当前结果，因此将其从本轮搜索中剪枝。对于仍可能包含更优点的单元，算法更新条件样本和变量条件景观，按照式 (26) 选择分裂变量与阈值，再对所得子单元重新传播约束、估计质量并计算上下界。当一个单元已经足够小，或继续分裂带来的界收紧有限时，算法才从中生成完整可行状态并计算点采集值。

记 $\hat{x}_t$ 为当前找到的最佳可行实验点。当所有未检查单元的最大采集上界与 $a_t(\hat{x}_t)$ 足够接近时，搜索停止：

$$\max_{U \in \mathcal{Q}_t} \bar{a}_t(U) - a_t(\hat{x}_t) \leq \varepsilon_t \quad (27)$$

其中 $\varepsilon_t \geq 0$ 是本轮允许的采集误差。式 (27) 表明， $\hat{x}_t$ 的采集值与完全枚举所得最优值之间至多相差 ε_t 。这一结论成立的前提是单元上下界有效；根据代理模型和质量估计方法不同，这些界可以是确定性界，也可以是带置信水平的概率界。这里的剪枝只表示本轮不再到该单元中寻找实验点，并不会把该单元从全局风险计算中删除。

5.5 新状态探索与已有状态复验

由于运行结果具有随机性，在同一个状态上只运行一次通常不足以判断其失效概率。因此，每一轮既可以探索尚未测试的新状态，也可以复验已有状态。探索的价值在于获得新区域的信息，并通过代理模型影响相关状态；复验的价值在于减少已有状态上的概率不确定性。两类动作都用单位成本下的预期后验风险下降评价。

为统一表示探索和复验，允许一次实验决策在状态 x_{t+1} 上安排 m_{t+1} 次独立运行：

$$(x_{t+1}, m_{t+1}) \in \mathcal{X} \times \{1, 2, \dots\}, \quad (28)$$

相应的批量采集值为

$$a_t(x, m) = \frac{R_t - \mathbb{E}[R_{t+m} \mid \mathcal{H}_t, \text{在 } x \text{ 处独立运行 } m \text{ 次}]}{m c(x)}. \quad (29)$$

分子是这 m 次运行预计减少的后验部署风险，分母 $m c(x)$ 是批量运行成本。较大的 m 可以减少重复计算采集函数的开销，但也可能在新结果尚未返回前过早锁定预算。实现时可以每运行一次就重新选点，也可以只允许较小批量。后文实验将比较固定复验次数、基于后验方差的复验和直接按部署风险下降决定的复验。

5.6 全局后验风险区间与任意时刻停止证书

采集搜索只需找到一个有价值的下一实验点，而风险报告必须始终覆盖整个可行域。为此，另用 $\mathcal{P}_t$ 表示当前的风险分割：其中各单元两两不相交，合起来恰好覆盖 $\mathcal{X}$ 。将所有单元的风险上下界分别相加，得到

$$\underline{R}_t = \sum_{U \in \mathcal{P}_t} \underline{R}_t(U), \quad \overline{R}_t = \sum_{U \in \mathcal{P}_t} \overline{R}_t(U). \quad (30)$$

因此， $[\underline{R}_t, \overline{R}_t]$ 包住当前的全局后验部署风险。当区间已经足够窄，或者风险上界已经低于业务允许值时，可以停止继续实验：

$$\overline{R}_t - \underline{R}_t \leq \delta_R \quad \text{或} \quad \overline{R}_t \leq R_{\max}. \quad (31)$$

其中， δ_R 是允许的风险区间宽度， $R_{\max}$ 是业务可接受的风险上限。若单元质量或后验界本身只在一定概率下成立，还需同时报告整个区间的置信水平。需要强调的是， $\mathcal{P}_t$ 始终覆盖完整可行域，用于风险计算； $\mathcal{Q}_t$ 只服务于当前一轮的实验点搜索。

5.7 理论分析

理论分析首先考察采集搜索是否可靠。在单元质量精确、单元界有效、搜索能够完整展开且剪枝只依据有效界的条件下，需要证明分支定界返回的 $\hat{x}_t$ 满足式 (27)。变量条件景观只改变节点的展开顺序和搜索量，不改变该证书的正确性。

其次，实际实现可能只得到近似单元质量和近似后验，因此需要分析这些误差如何传递到采集差距，并说明景观估计误差如何影响搜索量。最后，在模型可识别且实验保持充分探索的条件下，研究部署区域估计是否收敛，或能否得到预算结束时的部署后悔上界。每项结论都将明确限定适用的约束问题类型、代理模型结构和概率保证条件。

6 说明性示例：订单履约策略的部署边界

考虑一个订单履约系统。状态

$$x = (q, I_1, I_2, R_1, R_2, S, V, P) \quad (32)$$

包含订单量 q 、两个仓库的库存 I_i 、预留量 R_i 、是否允许拆单 S 、订单价值 V 与审批状态 P 。这里改用 P 表示审批状态，以免与第 3.3 节的选点策略 $\mathcal{A}$ 混淆。可行输入状态需要满足库存、审批和履约规则，例如

$$0 \leq R_i \leq I_i, \quad i = 1, 2, \quad (33)$$

$$q \geq 1, \quad (34)$$

$$V > V_0 \implies (S = 0 \text{ 或 } P = \text{已审批}), \quad (35)$$

$$q \leq (I_1 - R_1) + (I_2 - R_2). \quad (36)$$

前两项保证预留量和订单量有效；第三项表示高价值订单只有在不拆单或已经审批时才可进入相应流程；最后一项保证两个仓库的可用库存足以覆盖订单量。被测策略可以是启发式履约器、学习型调度策略或调用工具的 AI Agent。它需要在单仓发货、拆单、等待补货和升级审批等动作之间选择，轨迹判定器则检查最终履约结果以及库存和审批规则是否被违反。

假设自动策略通常比人工处理便宜，但在未获审批的高价值订单上错误拆单会造成较大损失。根据式 (6)，这类状态的失效损失 $\ell(x)$ 较大，因而可接受失效概率 $\alpha(x)$ 较低。实验的目标不是寻找一个最坏订单，而是判断哪些可行订单状态仍适合自动履约。

初始运行后，一个约束单元

$$U_1 = \{V > V_0, P = \text{未审批}, I_1 - R_1 < q\} \quad (37)$$

可能同时覆盖许多订单，并具有较高且不确定的后验部署风险。约束传播先收紧拆单和库存变量的可行范围，删除库存不足等不可能履约的组合。随后，C-SUR 在 U_1 中生成若干完整可行订单，比较采集值随订单价值、拆单选项和库存余量的变化。

如果高采集值主要出现在“总可用库存刚好接近订单量”的样本附近，而两个仓库各自库存的差异并不重要，变量条件景观就会优先建议按总库存余量分裂。分裂后，算法重新计算各子单元的质量和采集界，并剪去不可能包含更优实验点的子单元。

实验动作包括在未观察的边界输入状态运行，以及复验后验分布仍跨越 $\alpha(x)$ 的已有状态。算法按单位成本的期望全局后验部署风险下降比较两类动作。输出包括部署区域估计、风险区间和约束单元描述，例如

$$V > V_0, \quad P = \text{未审批}, \quad I_1 - R_1 < q \leq \sum_i (I_i - R_i). \quad (38)$$

该描述只表示统计性高风险关联；若要断言某一条件导致失效，还需要额外干预实验。

7 计算实验设计

7.1 研究问题

计算实验分别考察统计效果、搜索效率和可扩展性：

1. 在相同黑盒运行成本下, C-SUR 能否降低部署后悔和参考测度加权的区域误判?
2. 约束传播、单元质量、单元界和变量条件景观分别带来多少改进?
3. 在达到相同采集差距时, 景观引导分裂能否减少节点展开、完整状态生成和求解器调用?
4. 当运行成本随状态变化或运行噪声增大时, 按部署风险下降决定复验次数是否优于固定复验?
5. 当可行域从可枚举小实例扩大到固定候选池难以覆盖的组合空间时, 方法的统计效果和计算成本如何变化?

7.2 实例层次

实验分为三个层次。第一层使用能够完全枚举的合成约束问题, 包括受约束背包、配置和调度。由于可以对每个可行状态进行大量独立运行, 这些实例能够近似给出真实的 $p(x)$ 、最优部署区域 $\mathcal{D}^*$ 和枚举式 SUR 采集值, 用于检查部署后悔、采集差距和证书覆盖率是否计算正确。

第二层使用无法完全枚举的大型组合问题。此时不再要求获得逐状态真值, 而重点测量运行时间、内存、约束传播的剪枝比例、单元质量估计误差和单位黑盒预算下的效果。第三层使用具有实际决策过程的黑盒策略, 包括经典启发式或学习型策略, 以及由 Tapas 描述业务规则的 LLM Agent 环境, 用于检验方法在长轨迹和工具调用场景中的适用性。

7.3 实现

计算实验使用 Tapas 表达确定性业务约束, 并实现部分赋值下的增量传播、变量域查询和分支状态复制。外部模块根据传播结果完成单元质量估计和条件采样; 黑盒运行器执行被测策略并记录轨迹, 再由固定判定规则 F 生成标签。C-SUR 只通过上述传播接口访问可行域, 因此实验中的 Tapas 可以替换为提供同类接口的约束规划、SAT、混合整数规划或领域专用生成器。

7.4 基线和消融

主要基线是约束过滤候选池 SUR (CP-filtered SUR)。该方法先由约束求解器生成一个固定的可行候选池, 再在池内使用标准 SUR 选点。它与 C-SUR 使用相同的后验风险和黑盒预算, 因此二者的差异主要来自是否在实验过程中继续搜索候选池之外的可行状态。

为了判断改进究竟来自何处, 还设置三类辅助基线。按 μ 随机采样和覆盖驱动组合测试用于比较自适应风险选点与分布覆盖 [16]; 阈值邻近采样、点式贝叶斯优化和自适应压力测试用于比较部署后悔目标与边界或极值搜索目标 [2, 3, 10, 11]; 固定复验方法用于比较自适应复验分配 [7, 8]。分裂规则则分别采用固定变量顺序、最小域优先、质量平衡、后验方差和变量条件景观。

消融实验逐一关闭传播剪枝、单元质量、单元采集界、变量交互景观、自适应复验和部署成本权重, 并改变每个单元的景观样本量。这样可以区分各模块对统计效果和计算开销的贡献。

7.5 评价指标与公平性

统计效果以累计实验成本为横轴，报告部署后悔 (8) 曲线及其曲线下面积。辅助指标包括参考测度加权的假阴性和假阳性、全局风险区间宽度，以及最终允许自动部署的状态质量。

计算开销通过墙钟时间、峰值内存、求解器调用次数、节点展开数、完整可行状态生成数和剪枝比例衡量。对于单元质量和采集搜索，还分别报告质量估计误差以及式 (27) 给出的采集差距。变量条件景观额外报告自身计算成本，以及达到同一采集差距所需的搜索量。

所有方法使用相同的初始信息、黑盒运行成本预算和独立验证集。用于估计景观的条件样本没有执行黑盒策略，因此只计入计算开销，不计入实验历史或黑盒预算。每组实验使用多个随机种子，报告置信区间，并保存完整的预算轨迹、实例配置和失效判定规则，以便复现比较。

8 研究假设与适用范围

计算实验检验两个主要假设。第一个假设关注统计效果：在相同的黑盒运行总成本下，C-SUR 在完整可行域上的部署后悔低于约束过滤候选池 SUR。第二个假设关注搜索效率：在达到相同采集差距时，变量条件景观引导的分裂比通用约束规划分支规则展开更少的节点。

这两个优势并非在所有问题上都应当出现。实验将进一步考察它们如何随可行域稀疏度、单元内采集值差异、候选池遗漏质量和状态相关运行成本而变化。当可行域较小、各处采集值近似相同，或固定候选池已经覆盖充分时，计算变量条件景观的额外成本可能超过它减少搜索量所带来的收益。

本文报告的采集差距和风险区间依赖单元质量区间及单元上下界的有效性。变量条件景观只用于安排搜索顺序，不能替代这些界，也不能单独构成正确性保证。本文同样不声称能够为任意黑盒策略提供全局安全证明，统计上识别出的高风险区域也不能解释为失效的因果机制。

最后，结论依赖第 3.1 节的固定版本假设。判定规则发生变化会改变标签 Y ；被测策略或仿真环境发生变化会改变 $p(x)$ ；部署流量或管理权重发生变化会改变参考测度 μ 。计算实验将分别扰动这些因素，报告部署规则和风险区间对模型失配与分布变化的敏感性。

参考文献

- [1] Berger, J. O. (1985). *Statistical Decision Theory and Bayesian Analysis*, 2nd ed. Springer.
- [2] Gotovos, A., Casati, N., Hitz, G., and Krause, A. (2013). Active learning for level set estimation. *Proceedings of the 23rd International Joint Conference on Artificial Intelligence*, 1344–1350.
- [3] Bogunovic, I., Scarlett, J., Krause, A., and Cevher, V. (2016). Truncated variance reduction: A unified approach to Bayesian optimization and level-set estimation. *Advances in Neural Information Processing Systems*, 29.

- [4] Bect, J., Ginsbourger, D., Li, L., Picheny, V., and Vazquez, E. (2012). Sequential design of computer experiments for the estimation of a probability of failure. *Statistics and Computing*, 22:773–793.
- [5] Chevalier, C., Bect, J., Ginsbourger, D., Vazquez, E., Picheny, V., and Richet, Y. (2014). Fast parallel kriging-based stepwise uncertainty reduction with application to the identification of an excursion set. *Technometrics*, 56(4):455–465.
- [6] Lyu, X., Binois, M., and Ludkovski, M. (2021). Evaluating Gaussian process metamodels and sequential designs for noisy level set estimation. *Statistics and Computing*, 31:43.
- [7] Chen, C.-H., Lin, J., Yücesan, E., and Chick, S. E. (2000). Simulation budget allocation for further enhancing the efficiency of ordinal optimization. *Discrete Event Dynamic Systems*, 10:251–270.
- [8] Song, E. (2021). Sequential Bayesian risk set inference for robust discrete optimization via simulation. arXiv:2101.07466.
- [9] Au, S.-K. and Beck, J. L. (2001). Estimation of small failure probabilities in high dimensions by subset simulation. *Probabilistic Engineering Mechanics*, 16(4):263–277.
- [10] Corso, A., Moss, R. J., Koren, M., Lee, R., and Kochenderfer, M. J. (2021). A survey of algorithms for black-box safety validation of cyber-physical systems. *Journal of Artificial Intelligence Research*, 72:377–428.
- [11] Lee, R., Mengshoel, O. J., Saksena, A., et al. (2020). Adaptive stress testing: Finding likely failure events with reinforcement learning. *Journal of Artificial Intelligence Research*, 69:1165–1201.
- [12] Rossi, F., van Beek, P., and Walsh, T., eds. (2006). *Handbook of Constraint Programming*. Elsevier.
- [13] Régin, J.-C. (1994). A filtering algorithm for constraints of difference in CSPs. *Proceedings of the 12th National Conference on Artificial Intelligence*, 362–367.
- [14] Chakraborty, S., Meel, K. S., and Vardi, M. Y. (2013a). A scalable approximate model counter. *Proceedings of the 19th International Conference on Principles and Practice of Constraint Programming*, 200–216.
- [15] Chakraborty, S., Meel, K. S., and Vardi, M. Y. (2013b). A scalable and nearly uniform generator of SAT witnesses. *Proceedings of the 25th International Conference on Computer Aided Verification*, 608–622.
- [16] Wu, H., Nie, C., Petke, J., Jia, Y., and Harman, M. (2019). A survey of constrained combinatorial testing. arXiv:1908.02480.

- [17] Jin, H. and Tsuchiya, T. (2020). Constrained locating arrays for combinatorial interaction testing. *Journal of Systems and Software*, 170:110771.